2

HỌC MÁY
TĂNG CƯỜNG CHO
CÁC HỆ THỐNG MẠNG

Làm quen với thuật toán Q-Learning

Biên soạn: ThS. Phan Trung Phát
Liên hệ: phatpt@uit.edu.vn

Tái bản lần 1 - Tháng 03/2026

Lưu hành nội bộ

A. MỤC TIÊU

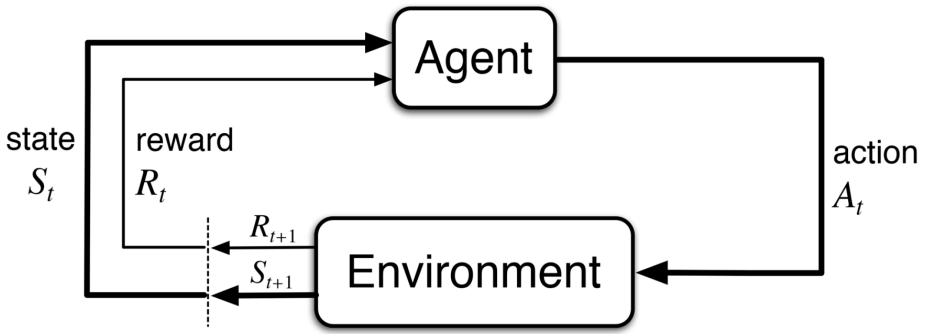
- Giới thiệu tổng quát phân loại các nhóm giải thuật học tăng cường.
- Làm quen với thuật toán Q-learning, cách hiện hiện thực và phân tích các trường hợp ứng dụng.
- Ứng dụng Q-learning vào giải quyết các bài toán cổ điển dựa trên môi trường được cung cấp sẵn và môi trường tự xây dựng, đặc biệt là bài toán Load Balancing.
- Thực hiện đánh giá và kiểm thử trên các chính sách.

B. GIỚI THIỆU

1. Sơ lượt về phân loại các giải thuật Reinforcement Learning

a. Model-based và Model-free Reinforcement Learning

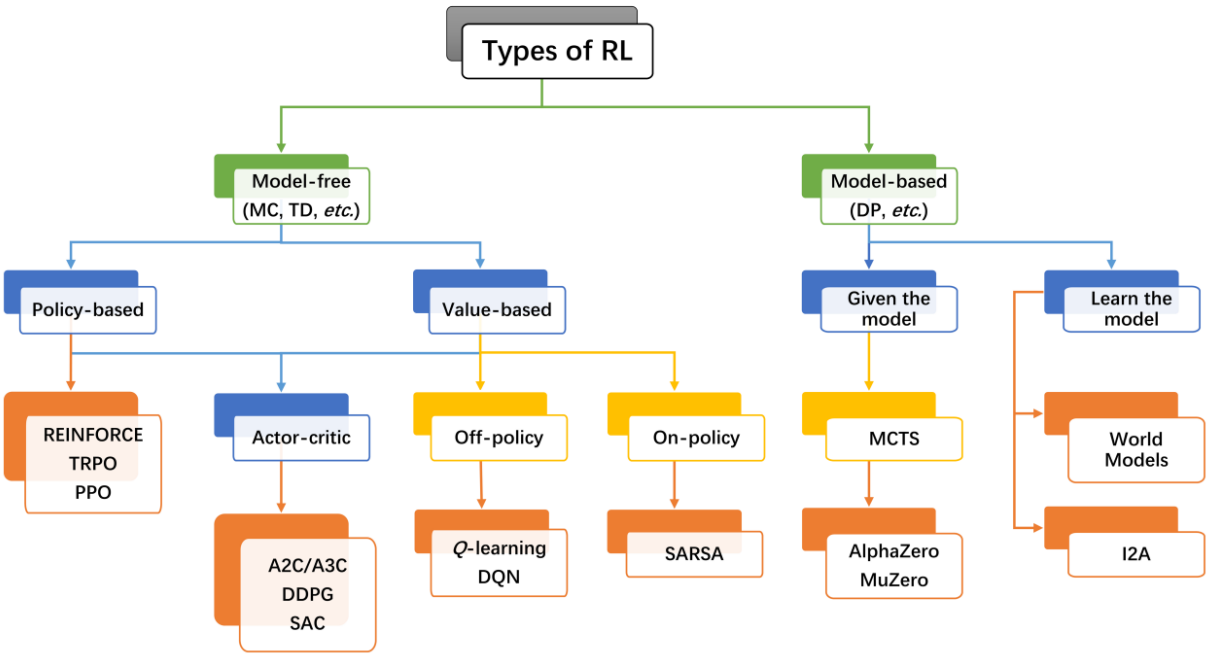
Trong **Học tăng cường** (Reinforcement Learning, RL), mục tiêu của một tác nhân (Agent) là học cách đưa ra quyết định tối ưu thông qua việc tương tác với môi trường (Environment). Trong quá trình này, Agent quan sát trạng thái của môi trường, thực hiện một hành động, sau đó nhận được phản hồi dưới dạng phần thưởng và trạng thái mới. Bằng cách lặp lại quá trình này nhiều lần, Agent dần học được hành vi nào mang lại lợi ích lâu dài lớn nhất. Vậy "*Agent cần hiểu môi trường ở ngoài đến mức độ nào để đưa ra quyết định đúng đắn?*"



Hình 1. Vòng lặp học tập của học tăng cường

Đối với RL, khi đứng trước một môi trường xa lạ đều phải lựa chọn một trong hai chiến lược sinh tồn: Hoặc là dành thời gian quan sát để xây dựng một bản đồ logic đầy đủ trong

đầu, hoặc là cứ dấn thân hành động để rút ra bài học từ những nỗi đau và phần thưởng trực tiếp. Hai hướng tiếp cận này cũng là hai nhánh chính dùng để phân loại các thuật toán RL: **Model-based** và **Model-free** Reinforcement Learning.



Hình 2. Phân loại các giải thuật RL[1]

*Đối với Model-based RL:

Ở đây, Agent cố gắng xây dựng hoặc đã biết trước "mô hình" của thế giới, bao gồm:

- **Xác suất chuyển trạng thái** $P(s'|s,a)$: Khả năng rơi vào trạng thái s' khi thực hiện hành động a tại s.
- **Hàm phần thưởng** $R(s,a)$: Biết trước sẽ nhận được bao nhiêu điểm.

Trong trường hợp Agent biết trước được quy luật xác định trạng thái nào sẽ xảy ra tiếp theo khi một hành động được thực hiện và có đầy đủ thông tin từ môi trường thì nó có thể dự đoán trước hậu quả của các hành động và lập kế hoạch để chọn chuỗi hành động tốt nhất.

[1] Liu X, Zhang J, Hou Z, Yang YI, Gao YQ. From predicting to decision making: Reinforcement learning in biomedicine. WIREs Comput Mol Sci. 2024;14(4):e1723. https://doi.org/10.1002/wcms.1723

Cách tiếp cận này giống như việc một người có bản đồ đầy đủ của một thành phố. Khi muốn đi từ điểm A đến điểm B, họ có thể phân tích tất cả các tuyến đường có thể và chọn ra con đường tối ưu trước khi bắt đầu di chuyển. Đây chính là cách các thuật toán **Model-based** hoạt động.

*Đối với Model-free RL:

Tuy nhiên, trong nhiều hệ thống thực tế, đặc biệt là các hệ thống phức tạp như mạng máy tính, việc biết chính xác mô hình của môi trường là rất khó. Lưu lượng mạng có thể thay đổi liên tục, độ trễ của các liên kết có thể biến động theo thời gian, và hành vi của người dùng thường không thể dự đoán chính xác. Trong những trường hợp như vậy, việc xây dựng một mô hình đầy đủ của môi trường trở nên không khả thi.

Chính vì lý do đó, **Model-free RL** trở thành một hướng tiếp cận quan trọng. Trong phương pháp này, Agent không cố gắng xây dựng hoặc sử dụng mô hình của môi trường. Thay vào đó, Agent học trực tiếp từ trải nghiệm (thử và sai). Mỗi lần thực hiện một hành động, Agent quan sát kết quả và sử dụng thông tin đó để cải thiện quyết định trong tương lai. Agent hoàn toàn không biết gì về luật lệ vật lý hay xác suất của môi trường. Nó giống như một người mù đi trong phòng tối, chỉ khi đụng tường mới biết có tường.

b. Value-based và Policy-based methods

Cho dù ở bất kỳ phương pháp nào, mục tiêu chính của RL là tìm ra một **policy** – tức là một chiến lược lựa chọn hành động dựa trên trạng thái hiện tại của môi trường – sao cho tổng phần thưởng kỳ vọng G_t là lớn nhất. Đặc biệt hơn là đối với phương pháp Model-free RL, việc lựa chọn hướng tiếp cận sẽ có thể quyết định tốc độ học, khả năng tính toán và độ chính xác của Agent trong nhưng môi trường phức tạp. Hai chiến lược cốt lõi để tìm ra giải pháp tối ưu trong nhóm này là: **Value-based** và **Policy-based** methods. Hãy tưởng tượng bạn đang huấn luyện một chú chó. Bạn có hai lựa chọn: Một là dạy nó hiểu "giá trị" của từng hành

động (ngồi xuống thì khả năng cao có bánh, chạy lung tung thì không có gì) để nó tự tính toán (Value-based methods); hai là huấn luyện nó thành một "phản xạ" thuần túy (hễ nghe tiếng huýt sáo là ngồi xuống ngay lập tức mà không cần suy nghĩ về phần thưởng) (Policy-based methods).

*Value-based methods:

Các Value-based methods dựa trên một ý tưởng khá trực quan: thay vì học ngay cách hành động, Agent trước tiên học cách đánh giá **mức độ tốt** của các trạng thái hoặc hành động. Khi đã có được sự đánh giá này, việc lựa chọn hành động tối ưu trở nên đơn giản hơn – Agent chỉ cần chọn hành động được đánh giá là có giá trị cao nhất.

Trong RL, việc đánh giá này được thực hiện thông qua việc xấp xỉ hàm giá trị $Q(s, a)$. Giá trị này đại diện cho tổng phần thưởng tích lũy kỳ vọng mà Agent sẽ nhận được từ tương lai, bắt đầu từ trạng thái s và thực hiện hành động a. Nguyên lý tối ưu Bellman được sử dụng và Agent tin rằng giá trị của hiện tại có thể được tính bằng phần thưởng tức thì cộng với giá trị tốt nhất của trạng thái kế tiếp. Mục tiêu tối cao của Agent là xấp xỉ được Hàm giá trị tối ưu $Q^*(s, a)$.

Nếu ta biết được giá trị $Q(s, a)$ cho mọi hành động có thể, policy tối ưu có thể được suy ra bằng cách chọn hành động có giá trị lớn nhất:

$$\pi(s) = argmax_{\mathrm{a}} Q(s, a)$$

Nói cách khác, Value-based methods học cách đánh giá trước, sau đó hành động dựa trên kết quả đánh giá đó.

*Policy-based methods:

Trong khi Value-based methods tập trung vào việc học giá trị của trạng thái hoặc hành động, Policy-based methods đi theo một hướng tiếp cận khác: học trực tiếp chiến lược hành động.

Trong cách tiếp cận này, policy thường được mô hình hóa như một hàm xác suất. Nghĩa là tại mỗi trạng thái, policy sẽ cho biết xác suất lựa chọn từng hành động có thể. Các tham số của policy (thường được biểu diễn bằng một mô hình học máy như neural network) sẽ được điều chỉnh sao cho các hành động dẫn đến phần thưởng cao trở nên có xác suất được lựa chọn lớn hơn.

Quá trình học thường được thực hiện bằng cách tối ưu hóa trực tiếp tổng phần thưởng kỳ vọng. Nói một cách đơn giản, nếu một chuỗi hành động mang lại kết quả tốt, thuật toán sẽ điều chỉnh policy để những hành động trong chuỗi đó có nhiều khả năng được chọn hơn trong các lần thử tiếp theo.

c. on-policy và off-policy

Ngoài cách phân loại dựa trên việc học value hay policy, các thuật toán RL còn được phân biệt dựa trên cách dữ liệu trải nghiệm được sử dụng trong quá trình học. Hai khái niệm quan trọng trong trường hợp này là **on-policy** và **off-policy** learning.

Để hiểu sự khác biệt này, cần phân biệt hai loại chính sách:

- **Chính sách Mục tiêu** (**Target Policy** - π): Là chính sách mà tác nhân đang cố gắng học tập hoặc cải thiện để trở nên tối ưu.
- **Chính sách Hành vi** (**Behavior Policy** - b): Là chính sách được sử dụng để tạo ra các hành động thực tế và thu thập trải nghiệm (dữ liệu) từ môi trường.

*On-policy learning:

Trong on-policy learning, Agent học dựa trên chính policy mà nó đang sử dụng để tương tác với môi trường. Điều này có nghĩa là dữ liệu được thu thập trong quá trình thực thi policy hiện tại cũng chính là dữ liệu được dùng để cập nhật policy đó.

$$\pi = b$$

Ưu điểm của cách tiếp cận on-policy là quá trình học thường ổn định hơn vì dữ liệu huấn luyện luôn phù hợp với policy hiện tại. Tuy nhiên, nhược điểm là dữ liệu cũ thường khó tái sử dụng, khiến việc học đôi khi kém hiệu quả về mặt dữ liệu.

*Off-policy learning:

Ngược lại, off-policy learning cho phép Agent học một policy trong khi dữ liệu được thu thập từ một policy khác. Điều này có nghĩa là quá trình học và quá trình thu thập dữ liệu có thể tách rời nhau.

$$\pi \neq b$$

Ưu điểm của off-policy learning là khả năng tái sử dụng dữ liệu hiệu quả. Các trải nghiệm trong quá khứ có thể được lưu lại và sử dụng nhiều lần trong quá trình huấn luyện. Điều này đặc biệt quan trọng trong các hệ thống RL hiện đại sử dụng **experience replay**.

2. Giới thiệu về thuật toán Q-learning

a. Giới thiệu và ý tưởng thuật toán

Trong số các thuật toán RL, **Q-learning** là một trong những phương pháp cơ bản và có ảnh hưởng lớn nhất. Thuật toán này thuộc nhóm Value-based methods, vì nó tập trung vào việc học giá trị của các hành động tại từng trạng thái. Đồng thời, Q-learning cũng là một thuật toán off-policy, nghĩa là policy được học có thể khác với policy được sử dụng để thu thập dữ liệu trong quá trình tương tác với môi trường.

Nhờ tính đơn giản và khả năng học trực tiếp từ trải nghiệm, Q-learning đã trở thành một trong những nền tảng quan trọng của RL và được sử dụng rộng rãi trong nhiều nghiên cứu cũng như ứng dụng thực tế.

Thuật toán Q-learning lần đầu được giới thiệu vào cuối thập niên 1980 bởi nhà nghiên cứu Christopher J. C. H. Watkins[2], trong quá trình nghiên cứu về các phương pháp RL không cần mô hình môi trường. Sau đó, công trình này được phát triển và phân tích sâu hơn trong luận án tiến sĩ của ông, với sự đóng góp thêm từ Peter Dayan.

Ý tưởng cốt lõi của Q-learning khá đơn giản. Thay vì cố gắng học toàn bộ mô hình của môi trường, Agent chỉ cần học một hàm giá trị gọi là **Q-function**, dùng để đánh giá chất lượng của từng hành động trong từng trạng thái. Nếu Agent biết chính xác giá trị Q của mọi **cặp trạng thái–hành động**, thì việc lựa chọn hành động tối ưu trở nên rất đơn giản: tại mỗi trạng thái, Agent chỉ cần chọn hành động có giá trị Q lớn nhất. Do đó, nhiệm vụ chính của thuật toán Q-learning là học được các giá trị Q tốt nhất thông qua trải nghiệm.

Trong RL, giá trị $Q(s, a)$ biểu diễn tổng phần thưởng kỳ vọng trong tương lai nếu Agent thực hiện hành động a tại trạng thái s, sau đó tiếp tục hành động theo policy tối ưu. Về mặt trực giác, giá trị này phản ánh câu hỏi:

"*Nếu ở trạng thái này và thực hiện hành động này, thì kết quả dài hạn có tốt hay không?*"

Nếu một hành động thường dẫn đến những trạng thái có phần thưởng cao trong tương lai, giá trị Q của hành động đó sẽ dần tăng lên trong quá trình học.

Trong Q-learning, các giá trị này được cập nhật dần dần sau mỗi lần Agent tương tác với môi trường. Khi Agent thực hiện một hành động và nhận được phần thưởng, thuật toán sẽ điều chỉnh giá trị Q hiện tại dựa trên thông tin mới vừa quan sát được.

[2] Watkins, C.J.C.H., Dayan, P. Q-learning. Mach Learn 8, 279–292 (1992). https://doi.org/10.1007/BF00992698

Trong quá trình học, Q-learning sử dụng quy tắc cập nhật dựa trên Temporal Difference (TD) có dạng:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma a' max Q(s',a') - Q(s,a)]$$

Trong đó:

- s là trạng thái hiện tại.
- a là hành động được thực hiện.
- r là phần thưởng nhận được.
- s' là trạng thái tiếp theo.
- α là learning rate.
- γ là hệ số chiết khấu.

Phần biểu thức trong ngoặc vuông thể hiện **sai số giữa giá trị hiện tại và giá trị ước lượng tốt hơn**, và thuật toán sẽ điều chỉnh giá trị Q theo hướng giảm sai số này.

b. Cách thức hiện thực thuật toán

Trong các môi trường có *không gian trạng thái và hành động tương đối nhỏ*, Q-learning thường được cài đặt dưới dạng một bảng gọi là **Q-table**.

Q-table là một cấu trúc dữ liệu lưu trữ giá trị của hàm $Q(s,a)$, trong đó mỗi phần tử biểu diễn giá trị của việc thực hiện hành động a tại trạng thái s. Trong quá trình học, các giá trị này sẽ được cập nhật dần dần dựa trên các trải nghiệm mà Agent thu thập được khi tương tác với môi trường.

Nhờ cơ chế này, Agent có thể từng bước cải thiện hiểu biết của mình về môi trường và học được chiến lược hành động tốt hơn theo thời gian

Cách biểu diễn **Q-table**, giả sử môi trường có:

- N trạng thái.
- M hành động có thể thực hiện.

Khi đó Q-table có thể được biểu diễn dưới dạng một ma trận kích thước N x M:

State	Action 1	Action 2	Action 3
s1	Q(s1,a1)	Q(s1,a2)	Q(s1,a3)
s2	Q(s2,a1)	Q(s2,a2)	Q(s2,a3)
s3	Q(s3,a1)	Q(s3,a2)	Q(s3,a3)

Mỗi giá trị trong bảng thể hiện mức độ kỳ vọng của phần thưởng trong tương lai nếu Agent thực hiện hành động tương ứng tại trạng thái đó. Ban đầu, khi Agent chưa có thông tin về môi trường, các giá trị trong Q-table thường được khởi tạo bằng 0 hoặc một giá trị ngẫu nhiên nhỏ.

Quá trình học của Q-learning thường được thực hiện qua nhiều episode, trong đó mỗi episode là một chuỗi các bước tương tác giữa Agent và môi trường. Các bước chính của thuật toán có thể được mô tả như hình sau:

Algorithm 1. Q-Learning algorithm

Data: α, γ, ϵ
Result: Q-table
while *true* **do**
  (1) initialize $t \leftarrow 0$
  (2) start with S_0
  (3) at time t, use ε-greedy, pick

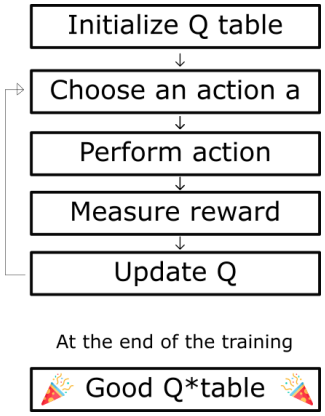
$$A_t = \begin{cases} \text{a random action,} & \text{with probability } \epsilon. \\ argmax_{a \in \mathcal{A}} Q(S_t, a), & \text{otherwise.} \end{cases}$$

  (4) perform A_t, observe R_{t+1}, and get into S_{t+1}
  (5) update $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_{a \in \mathcal{A}} Q(S_{t+1}, a) - Q(S_t, A_t))$
  (6) set $t \leftarrow t + 1$ and repeat step 3
end

Hình 3. Q-learning algorithm pseudocode[3]

[3] Le, N. T. T. (2023). Multi-agent reinforcement learning for traffic congestion on one-way multi-lane highways. Journal of Information and Telecommunication, 7(3), 255–269. https://doi.org/10.1080/24751839.2023.2182174

Hoặc có thể hiểu đơn giản hoá bằng hình ảnh sau:



Hình 4. Đơn giản hoá các bước thuật toán Q-learning[4]

Để triển khai Q-learning, cần xác định một số tham số quan trọng:

1. Learning rate (α): quyết định **mức độ ảnh hưởng của thông tin mới đến giá trị Q hiện tại**. Nếu giá trị này quá lớn, việc học có thể trở nên không ổn định. Nếu quá nhỏ, quá trình học sẽ diễn ra rất chậm.

2. Discount factor (γ): xác định **mức độ quan trọng của phần thưởng trong tương lai**. Nếu γ gần 1, Agent sẽ quan tâm nhiều đến phần thưởng dài hạn. Nếu γ nhỏ, agent sẽ ưu tiên phần thưởng ngắn hạn.

3. Exploration rate (ε): Trong quá trình học, Agent cần cân bằng giữa hai hoạt động:

- **Exploration**: thử các hành động mới để khám phá môi trường.
- **Exploitation**: chọn hành động tốt nhất dựa trên kiến thức hiện tại.

Một chiến lược phổ biến để thực hiện điều này là **ε-greedy policy**:

- với xác suất ε: chọn hành động ngẫu nhiên.
- với xác suất $1 - \varepsilon$: chọn hành động có giá trị Q lớn nhất.

[4] https://www.freecodecamp.org/news/diving-deeper-into-reinforcement-learning-with-q-learning-c18d0db58efe/

Chiến lược này giúp Agent vừa khám phá môi trường vừa tận dụng kiến thức đã học.

C. THỰC HÀNH

1. Hiện thực Q-learning với môi trường "Frozen Lake"

Mô tả: Sinh viên thực hiện tìm hiểu môi trường sẵn có "Frozen Lake" được cung cấp bởi Gymnasium để tiến hành hiện thực thuật toán Q-learning.

- **Bước 1:** Nhìn nhận bài toán.

Frozen Lake là một bài toán điều hướng trên lưới ô vuông điển hình, mô phỏng hành trình của một nhân vật băng qua mặt hồ đóng băng để đi từ điểm xuất phát (**S**) đến đích (**G**). Cấu trúc của mặt hồ được tạo thành từ các ô băng an toàn (**F**) đan xen với các hố nước nguy hiểm (**H**), nơi mà nếu Agent vô tình bước vào, lượt chơi sẽ kết thúc ngay lập tức với thất bại. Đây là một môi trường rời rạc hoàn hảo cho việc học máy tăng cường vì không gian trạng thái được mã hóa đơn giản bằng các số nguyên (ví dụ: từ 0 đến 15 cho lưới 4x4) và không gian hành động chỉ bao gồm bốn lựa chọn di chuyển cơ bản là Trái, Xuống, Phải và Lên..

- **Bước 2:** Tìm hiểu về các biểu diễn Markov Decision Process (MDP) của bài toán.
  - ? *Observation Space* mô tả điều gì?
  - ? *Action Space* mô tả điều gì?
  - ? *Reward* được định nghĩa như thế nào?
  - ? Nguyên tắc hoạt động của nhân vật?
  - ? Khi nào thì bài toán kết thúc?
- **Bước 3:** Cài đặt thư viện liên quan: `pip install -r requirements.txt`

- **Bước 4:** Mở file **Lab2.ipynb**, thực hiện nạp môi trường được cung cấp sẵn.

```
# Create deterministic FrozenLake environment for training
env = gym.make("FrozenLake-v1", is_slippery=False)

# Initialize Q-learning agent with default hyperparameters
agent = QLearningAgent(env)
```

- **Bước 5:** .Mở file **Lab2.ipynb**, hoàn thành các block code còn thiếu.

```
# Main training loop
for _ in range(num_episodes):
        # Reset environment at start of each episode
        state, _ = env.reset()
        done = False

        # Roll out one episode until terminal state
        while not done:
            # Choose action with exploration
            action = agent.choose_action(state, epsilon=0.1)

            # Execute action in environment
            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            # Update Q-table from transition (s, a, r, s')
            agent.update_q_value(state, action, reward, next_state)

            # Advance to next state
            state = next_state
```

```
def get_q_value(self, state, action):
        # Return Q(s, a). If missing, return 0.0 (default for unseen pairs).
        return self.q_table.get((state, action), 0.0)
...

def choose_greedy_action(self, state):
        # Pure greedy policy (epsilon=0):
        # Always choose one of the best actions by Q-value.
        q_values = [self.get_q_value(state, a) for a in range(self.env.action_space.n)]
        max_q = max(q_values)
        best_actions = [a for a, q in enumerate(q_values) if q == max_q]
        return random.choice(best_actions)
```

- **Bước 6:** Quan sát kết quả thu được.

? Hãy thử suy nghĩ việc hiện thực Q-learning trên môi trường **Mountain Car** như ở bài thực hành số 1 có thể gặp phải những rào cản nào? Có thể giải quyết bằng cách gì?

D. YÊU CẦU & NỘP BÀI

1. Yêu cầu

1. *Đối với FrozenLake Environment,* hoàn thành đoạn mã định nghĩa Q-learning còn thiếu trong file **Lab2.ipynb, Phần 1**. Đồng thời, thêm 1 lựa chọn để có thể thực hiện chính sách **Random** trong hàm `choose_action` và thực hiện chạy với cùng episode với optimal policy được chọn ra từ thuật toán Q-learning. Lưu trữ lại Reward qua từng episode, vẽ biểu đồ chồng giữa chính sách dựa trên Q-learning và Random. Từ đó, bạn có đánh giá gì đối với từng loại policy ở trên, giải thích, minh chứng.

Gợi ý: sử dụng thư viện Matplotlib.

2. *Đối với VacuumCleaner Evironment*, dựa trên các đoạn mã được cung cấp tại câu 1 với FrozenLake Environment, chỉnh sửa lại môi trường để có khả năng tích hợp với Q-learning và tiến hành hiện thực Q-learning trong file **Lab2.ipynb, Phần 2**. Hiện thực chính sách **Round Robin** và **Priority-based** tương tự tại bài thực hành số 1 trên môi trường đã chỉnh sửa.

3. Sau khi huấn luyện với Q-learning, thực hiện chạy môi trường với chính sách dựa trên Q-learning, Round Robin và Priority-based trong 100 episode. Viết hàm để tổng hợp Reward nhận được dưới dạng file .csv, vẽ biểu đồ thể hiện Reward đạt được theo thời gian. Bạn có đánh giá gì trên các giải pháp đã hiện thực và kết quả đạt được.

4. Thực hiện huấn luyện lại Q-learning với 3 trường hợp, tương ứng với 3 trọng số (Learning Rate, Discount Factor, Exploration Rate):

- **Trường hợp 1**: (0.5, 0.2, 0.1)
- **Trường hợp 2**: (0.1, 0.95, 0.2)
- **Trường hợp 3**: (0.1, 0.8, 0.7)

Hãy đánh giá về tốc độ hội tụ trong từng trường hợp, Reward đạt được. Từ đó, bạn có đánh giá gì trên 3 nhóm trọng số trên?

5. *Đối với LoadBalancing Evironment*, chỉnh sửa lại môi trường để có khả năng tích hợp với Q-learning và tiến hành hiện thực Q-learning trong file **Lab2.ipynb, Phần 3**. Hiện thực chính sách Round Robin tương tự tại bài thực hành số 1 trên môi trường đã chỉnh sửa.

6. Sau khi huấn luyện với Q-learning, thực hiện chạy môi trường với chính sách dựa trên Q-learning, Round Robin trong 100 episode. Viết hàm lưu trữ các metrics vào file *.csv*, các metrics cần lưu bao gồm: tổng thời gian chạy, tổng tác vụ đã tạo ra, số tác vụ được tiếp nhận để xử lý, số lượng tác vụ bị bỏ qua, số lượng tác vụ đã hoàn thành, phần thưởng.

7. Từ các số liệu thu thập được, viết hàm tính để tổng hợp các metrics vào điền vào bảng so sánh bên dưới:

	Q-learning	Round Robin
Average Reward		
Drop Rate		
Completion Rate		

Vẽ biểu đồ thể thiện sự biến thiên của từng policy theo thời gian với 3 metrics: reward, drop, completion. Bạn có đánh giá gì đối với từng loại policy ở trên, giải thích, minh chứng.

2. Yêu cầu nộp bài

- Sinh viên tìm hiểu và thực hành theo hướng dẫn. Thực hiện **theo nhóm**.
- Sinh viên báo cáo kết quả thực hiện và nộp bài bằng file. Trong đó:
  - Trình bày chi tiết quá trình thực hành và trả lời các câu hỏi nếu có (kèm theo các ảnh chụp màn hình tương ứng).
  - Giải thích các kết quả đạt được.
  - Tải mẫu báo cáo thực hành và trình bày theo mẫu được cung cấp.

Nén tất cả các file và đặt tên file theo định dạng theo mẫu: **NhomY-LabX_MSSV1_MSSV2** Ví dụ: Nhom1-Lab02_29520001_29520002_29520003

  - Nộp báo cáo trên theo thời gian đã thống nhất tại website môn học.
  - Các bài nộp không tuân theo yêu cầu sẽ **KHÔNG** được chấm điểm.

HẾT

Chúc các bạn hoàn thành tốt bài thực hành!